

BZAN 535: Statistical Methods

Joseph Falcone

2026-09-17

Table of contents

Preface	4
1 Role of Statistics and Key Concepts	5
1.1 Overview	5
1.2 Types of Data	5
1.3 Population vs. Sample	6
1.4 Two Broad Branches of Statistics	6
1.5 Statistical Tools	7
1.6 Study Designs: Observational vs. Experimental	7
2 Descriptive Analysis	9
2.1 Overview	9
2.2 Center	10
2.2.1 Mean	10
2.2.2 Median	10
2.3 Spread	11
2.3.1 Range	11
2.3.2 IQR	11
2.3.3 Variance	12
2.3.4 Standard Deviation	12
2.3.5 Coefficient of Variation	13
2.4 Shape	13
2.4.1 Symmetric	13
2.4.2 Right Skewed	13
2.4.3 Left Skewed	14
2.4.4 Outliers	14
2.5 Frequency Tables	14
2.6 Visualizations	14
2.6.1 Histograms	15
2.6.2 Boxplots	16
2.6.3 Scatterplots	17
2.6.4 Bar Charts	18
2.6.5 Pie Charts	19
2.7 ggplot2	20
2.7.1 Aesthetics	20

2.7.2	Layers	21
2.7.3	Mapping vs. Setting	22
2.7.4	Scales	24
2.7.5	Facets	25
2.7.6	Labels	26
2.7.7	Themes	27
3	Descriptive Analytics for Time Series Data	29
3.1	Overview	29
3.2	Working with Lubridate	29
3.2.1	Parsing Dates	30
3.2.2	Calendar Arithmetic	33
3.2.3	Time Windows	33
3.2.4	Seasonality	36
3.2.5	Moving Averages	38
4	Probability Rules and Distributions	40
5	Lecture 5	41
6	Lecture 6	42
7	Lecture 7	43
8	Lecture 8	44

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Role of Statistics and Key Concepts

1.1 Overview

- A decision before definitions
- Role of statistics in business
- Data types and measurement scales
- Populations vs. Samples
- Parameters vs. Statistics
- Study Designs: Observational vs. Experimental

Statistics: Business Question → Data → Analysis → Decision

The goal is to make that pipeline rigorous, not a simple “gut call.” There should be some method behind the process.

- Precisely describe what the data shows. *What was the average order value?*
- Estimate what’s outside the data with honest uncertainty. *What does our sample tell us about customers in general?*
- Compare groups within data and decide if differences are real or noise. *Do customers exposed to different policies actually behave differently?*
- Model relationships and determine if they’re causal. *What variables are related to spending and do they actually cause some sort of change?*

1.2 Types of Data

Categorical: qualitative values relating to characteristics.

- **Nominal:** categories with **no meaningful order**. There’s no mathematical sense in which is greater or less than another. *e.g. red, blue, green*
- **Ordinal:** categories **with an order** to them. Certain categories correspond to higher or lower rankings, places, etc. *e.g. one star, two star, three star*

Numerical

- **Interval:** numerical values with meaningful differences but no true zero. A value of zero doesn't necessarily mean "none", rather it's just a 'label' similar to any other number. *e.g. $0^{\circ}F$, $53^{\circ}F$*
- **Ratio:** numerical values with a true equal spacing and a true zero. In this case, a value of zero literally means "none" or "nothing there". *e.g. \$0, \$5* Because zero has an actual meaning, comparisons (or ratios) like "twice as much" are relevant.

1.3 Population vs. Sample

- Population: entire group of interest. This is the true group in question.
- Sample: the limited subset being observed.
- Parameter: a description of the population. Because it involves the entire group of interest, it's unknown (it's impossible to actually include everyone).
- Statistic: a computed description of the sample. Because the sample is only a part of the group, meaning it's an inference that it represents the true group, it's a predicted value.

	Population	Sample
Described by	Parameter	Statistic
Center	μ	$\bar{x}$
Proportion	p	$\hat{p}$
Spread	σ	s

Say in reality there's 50,000 customers and you survey 40. The average satisfaction score of the survey is 4.2. What you really want to know is $\mu = ?$ – the average satisfaction of all 50,000 customers.

Because only 40 were surveyed, you're using statistic $\rightarrow$ estimate parameter to estimate μ . That's the foundation of **statistical inference**.

Taste a spoon of soup (sample) to judge the pot (population), but only if you stirred it first (representative sample).

1.4 Two Broad Branches of Statistics

Descriptive vs. Inferential – that distinction becomes important.

Descriptive: describe the data actually observed. Things like mean, median, standard deviation, histogram, boxplot.

The customers in our sample spent an average of \$75.

That's descriptive

Inferential: use your sample to say something about a larger population. Things like confidence intervals, hypothesis tests, regression, experiments.

`*Based on the sample, we estimate RetailCo customers spend...*`

Assuming reality: inferential.

Goal	Tools	Output Examples
Describe Infer / Predict	Tables, histograms, boxplots Confidence intervals, hypothesis tests, regression, machine learning	Center, spread, shape Effects, forecasts

1.5 Statistical Tools

Question	Data	Tool
What's typical/variable?	Quantitative	Mean, SD, IQR, histogram, boxplot
How many are in each group?	Categorical	Frequency table, bar chart
Do groups differ?	Categorical + quantitative	Side-by-side boxplots, summaries
Are two quantities associated?	Two quantitative variables	Scatterplot, correlation

So you don't just randomly choose the statistical technique, the business question and variable types determine the appropriate tool.

1.6 Study Designs: Observational vs. Experimental

Suppose a company claims customers who use our app spend 40% more. Does that mean our app causes customers to spend more? Not necessarily. Many people who download the app could already be the company's most loyal customers. This is the problem of **confounding**.

Observational: observe what people already chose/did. Here, you can establish an association, but causality isn't established merely by observing a difference.

Experiment (A/B): random assignment of subjects. This randomization helps balance other characteristics (lurking variables) between groups. Therefore, differences in outcomes can support a causal interpretation, assuming everything else in the experiment is valid.

Randomized experiments can support cause-and-effect inference, whereas observational studies do not by themselves; random sampling is separately what supports inference to a population.

Random Sampling and random assignment are **not** the same thing.

- Random sampling answers “Can I generalize this result to the population?”.
- Random Assignment answers “Can I make a causal claim?”

Concept	What Is Randomized?	When?	Purpose
Random Sampling	Which people from the population are selected for the sample	When selecting a sample	Makes the sample more representative of the population and supports generalizing to the population
Random Assignment	Which treatment/group participants receive	When conducting an experiment	Makes treatment groups more comparable and supports stronger causal conclusions

2 Descriptive Analysis

2.1 Overview

Raw data can be messy and overwhelming, making it difficult to interpret and work with. Descriptive analysis helps solve this problem by summarizing and exploring data in a way that turns raw numbers into patterns and insights that can be understood accurately and easily.

There are three key questions to consider:

- What's typical?
- How spread out is it?
- What shape does it have?

How do we answer these questions? By examining the center, spread, and shape of the data – each of which describes a different aspect of the observations and together, helps us draw accurate conclusions about what the data is telling us.

Descriptive Analysis is organized around several key ideas:

- Center — What is typical?
- Spread — How much does the data vary?
- Shape — What does the distribution look like?
- Groups — How do different categories or populations compare?
- Visualization — How can we make patterns easier to see?

The first three, center, spread, and shape, describe the distribution itself. Groups lets you compare distributions, while visualization is how you explore and communicate those characteristics. And all are equally important.

A Quick Example: Suppose two products both sell an average of 100 units per week:

- Product A: 98, 101, 102, 99, 100 → extremely consistent
- Product B: 20, 180, 40, 160, 100 → highly variable

Both products have the same average, but their weekly sales behave very differently. Knowing the average alone does not necessarily tell us what sales actually look like.

They have the same center, but very different spread.

Together, these tools help answer the key questions needed to understand a dataset before moving on to deeper analysis.

2.2 Center

What's a typical value?

Center is the measure that represents the typical or central value of a dataset. It describes the location around which the data tends to lie.

There are two main measures of center: mean and median.

2.2.1 Mean

The **mean** is the arithmetic average of a dataset. It is calculated by adding all observations together and dividing by the number of observations.

$$\bar{x} = \frac{\sum x_i}{n}$$

Pros:

- Uses every observation in dataset.
- Easy to calculate and interpret.

Cons:

- Sensitive to outliers.
- Can be misleading for highly skewed distributions.

2.2.2 Median

Median is the middle value of a dataset after the observations have been ordered. It is more robust to outliers than the mean. For a odd number of observations, sort the data and take the middle value. For an even number of observations (i.e. two middle values) take the two middle values and average them.

Pros:

- Robust to outliers and extreme values.
- Often better represents the center of skewed data.

Cons:

- Does not incorporate the magnitude of every observation the same way the mean does.
- Can ignore meaningful differences in the values away from the middle.

2.3 Spread

How much do values vary?

Spread describes how much the observations in a dataset vary or differ from one another. In other words, it tells us how dispersed the data is around its center.

2.3.1 Range

The range measures the distance between the smallest and largest observations.

$$\text{range} = \text{maximum} - \text{minimum}$$

Pros:

- Extremely simple to calculate or understand.

Cons:

- Only considers two observations – the largest and smallest – so it is highly sensitive to outliers.

2.3.2 IQR

The **IQR** measures the spread of the middle 50% of the observations. In other words, how much do the middle 50% of values span? Because it ignores the extreme ends, it's resilient to outliers.

$$\text{IQR} = Q_3 - Q_1$$

Q_1 is the 25th percentile. Q_3 is the 75th percentile.

Conceptually, IQR looks at:

$$\underbrace{\text{lowest 25\%}} \quad | \quad \underbrace{Q_1 \leftrightarrow Q_3}_{\text{middle 50\%}} \quad | \quad \underbrace{\text{highest 25\%}}$$

Pros:

- Robust to outliers.

Cons:

- Does not incorporate all observations.

2.3.3 Variance

Variance measures how spread out observations are around the mean using their squared deviations from the mean. The deviations are squared so that negative and positive deviations do not cancel each other out.

To calculate:

1. Find the mean.
2. Find how far each observation is from the mean.
3. Square each difference.
4. Add the squared differences.
5. Divide by $n-1$.

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$$

Pros:

- Uses every observation in the dataset.
- Measures spread specifically around the mean, so it pairs naturally with the mean.

Cons: - Expressed in squared units, meaning it is less intuitive.

2.3.4 Standard Deviation

Standard Deviation is the square root of the variance. It allows for an interpretable scale of how much values differ from mean by putting variance back into original units.

$$s = \sqrt{s^2}$$

Pros:

- Easy to interpret.

Cons:

- Its magnitude depends on the scale of the data and can be misleading when comparing two variables of different scales.
- Sensitive to outliers.

2.3.5 Coefficient of Variation

The **Coefficient of Variation** measures the standard deviation relative to the mean. In other words, it tells you how large the standard deviation is compared to the mean (how big is distance based on mean).

$$CV = \frac{SD}{\bar{x}}$$

Pros:

- Can accurately compare spread of variables with different scales.

Cons:

- Not useful when mean is zero, because you cannot divide by zero.
- Can be sensitive to outliers.

2.4 Shape

What does the distribution look like?

Shape describes the overall pattern or form of a distribution. It helps us understand how observations are distributed around the center, including whether the data is symmetric, skewed, or contains outliers.

2.4.1 Symmetric

A **symmetric** distribution has approximately the same shape on both sides of its center. Neither side has a substantially longer tail than the other.

In a symmetric distribution, the mean and median tend to be relatively close.

2.4.2 Right Skewed

A **right-skewed** distribution has a long tail extending to the right, toward larger values.

A relatively small number of large observations stretch the distribution to the right and can pull the mean upward.

Typically: right skewed $\rightarrow$ mean $>$ median

2.4.3 Left Skewed

A **left-skewed** distribution has a long tail extending to the left, toward smaller values.

A relatively small number of small observations stretch the distribution to the left and can pull the mean downward.

Left tail → mean pulled left → Mean < Median Right tail → mean pulled right → Mean > Median

2.4.4 Outliers

An **outlier** is an observation that is unusually far from the rest of the data.

The **IQR rule** can be used to identify potential outliers.

lower bound = $Q_1 - 1.5(IQR)$

upper bound = $Q_3 + 1.5(IQR)$

If the observation is outside of these bounds it can be flagged as a potential outlier.

An outlier should not automatically be deleted. It is important to investigate its cause first. For example, an extreme observation could represent a data-entry error, a legitimate unusual observation, or a measurement/unit issue.

2.5 Frequency Tables

A **frequency table** summarizes a **categorical** variable by showing the count and/or proportion of observations in each category. It helps identify common and rare categories and understand the overall category mix.

Department	Count	Percent
Grocery	39,021	42.3%
Drug GM	31,529	34.1%
Produce	3,118	3.4%

2.6 Visualizations

Plot	Data	Main Question
Histogram	1 quantitative variable	What does the distribution look like?
Boxplot	Quantitative, often by category	How does the distribution/spread compare?
Scatterplot	2 quantitative variables	Are the variables related?
Bar chart	Categorical	How do category counts/proportions compare?
Pie chart	Categorical proportions	How is the whole divided among categories?

2.6.1 Histograms

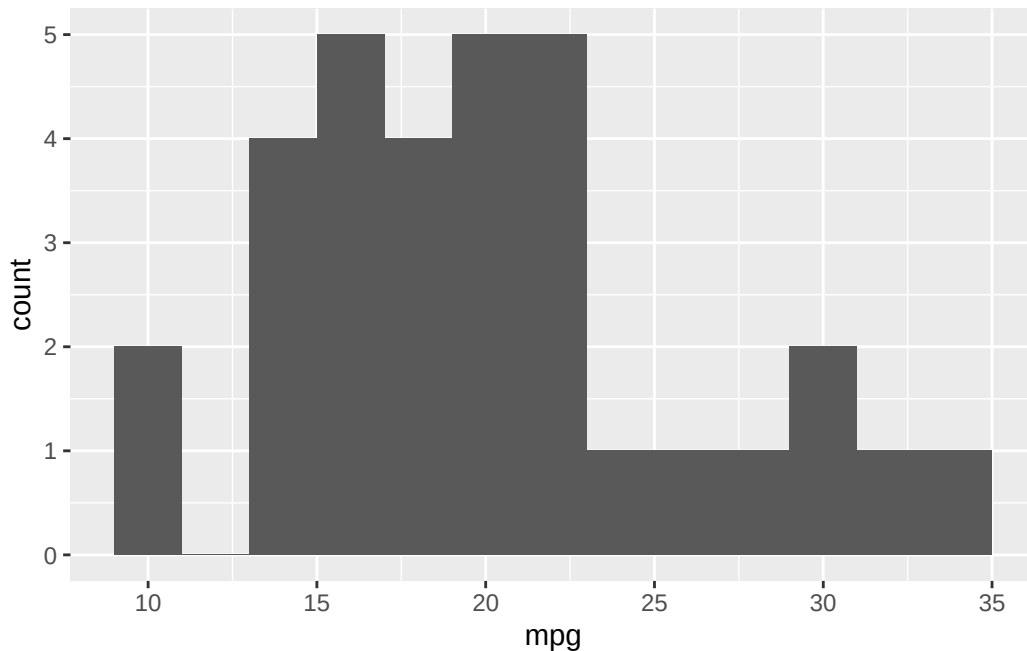
A **histogram** shows the **distribution** of a **single quantitative variable** by grouping numerical values into intervals called bins and showing how many observations fall into each bin.

Useful for identifying:

- Shape of a distribution
- Skewness
- Modes or peaks
- Gaps

The bin width determines how wide each interval is and therefore how much detail the histogram shows

```
ggplot(mtcars, aes(x = mpg)) +  
  geom_histogram(binwidth = 2)
```



Use when: You want to see what the distribution of one quantitative variable looks like.

2.6.2 Boxplots

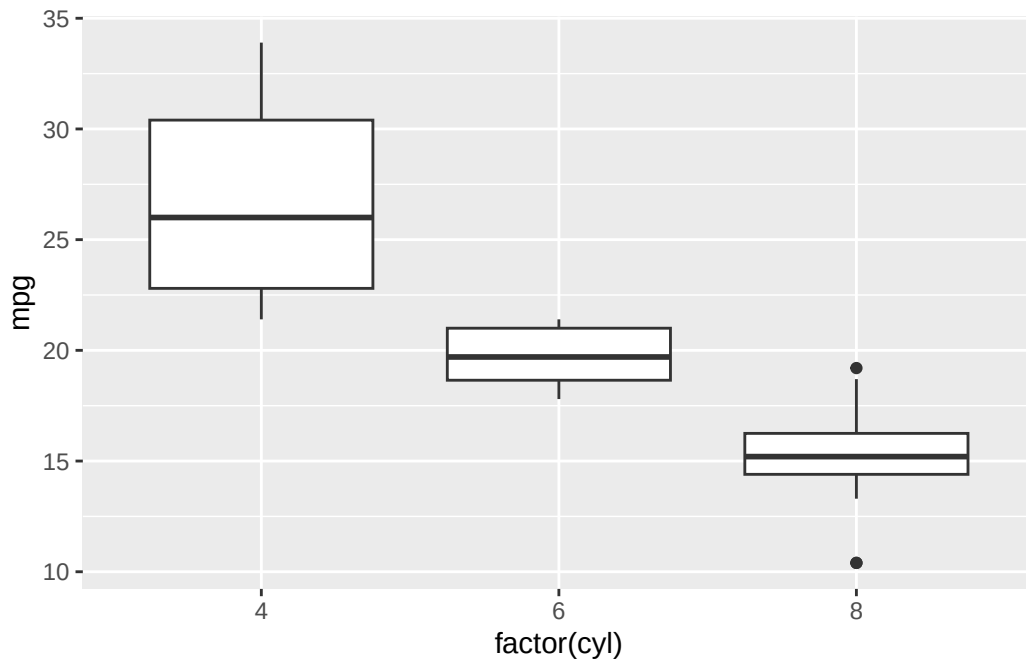
A **boxplot** provides a compact summary of the **distribution** of a **quantitative** variable using the five-number **summary**: Min, Q_1 , Median, Q_3 , Max

It is useful for examining:

- Center
- Spread
- Skewness
- Potential outliers
- Differences between groups

Boxplots are especially useful when you want to **compare distributions** across categories.

```
ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +  
  geom_boxplot()
```



Use when: You want to examine or compare the distributions of quantitative data, particularly across groups.

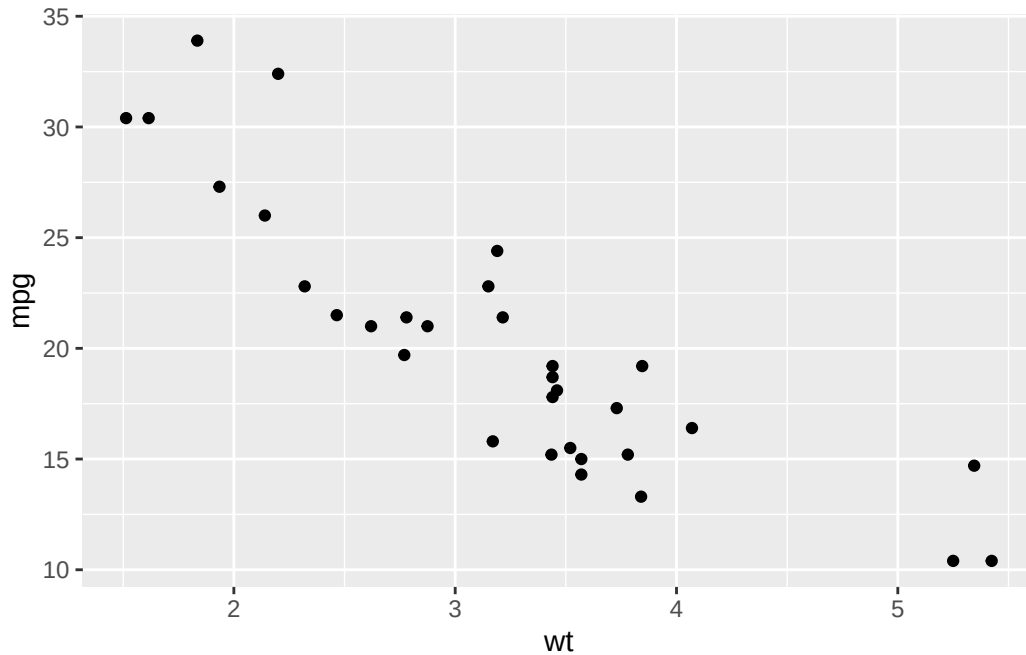
2.6.3 Scatterplots

A **scatterplot** shows the **relationship** between **two quantitative variables**. Each point represents one observation.

Useful for identifying:

- Direction — positive or negative relationship
- Strength — how closely the variables are related
- Form — linear, nonlinear, etc.
- Outliers

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point()
```



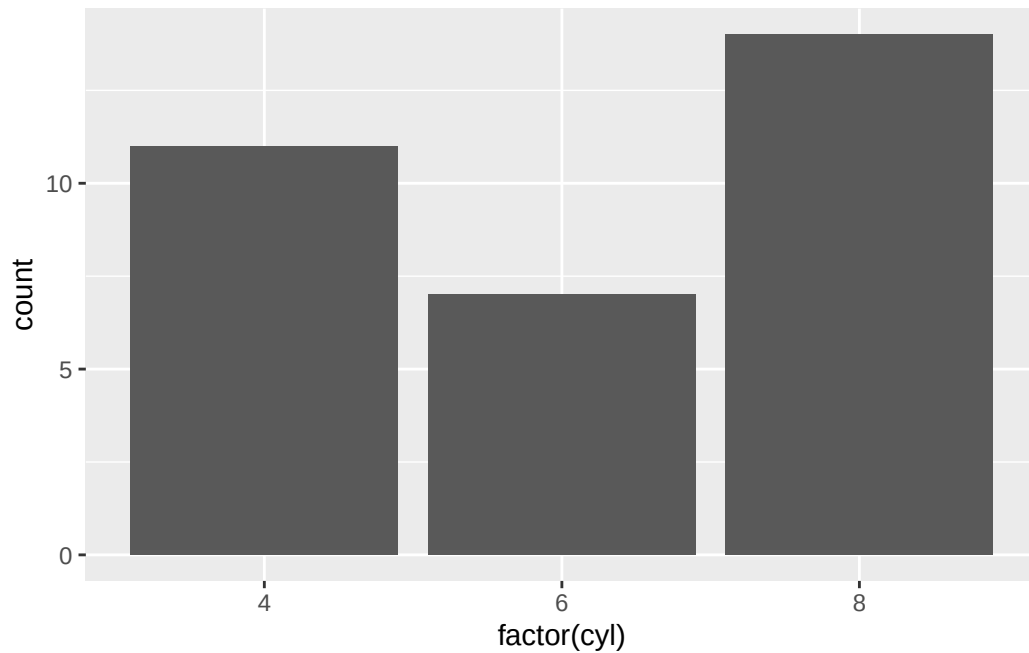
Use when: You want to know whether two quantitative variables are related.

2.6.4 Bar Charts

A **bar chart** displays the counts or proportions of categories for a categorical variable. It is essentially the graphical version of a **frequency table**.

The categories are represented by bars, and the height of each bar represents its count.

```
ggplot(mtcars, aes(x = factor(cyl))) +  
  geom_bar()
```



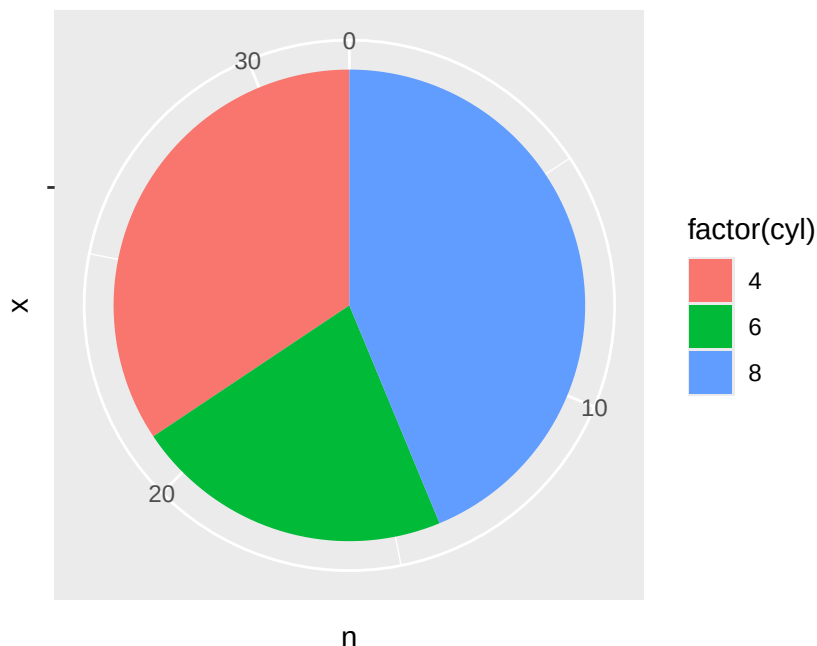
Use when: You want to compare counts or proportions across categories.

2.6.5 Pie Charts

A **pie chart** displays **proportions of a whole** as slices of a circle. Larger slices represent larger proportions.

Pie charts work best when there are only a **small number of categories**. Similar-sized slices can be difficult to compare visually, so a bar chart is often easier to interpret.

```
mtcars |>
  count(cyl) |>
  ggplot(aes(x = "", y = n, fill = factor(cyl))) +
  geom_col(width = 1) +
  coord_polar(theta = "y")
```



Use when: You want to show how a small number of categories make up a whole.

2.7 ggplot2

ggplot2 is a data visualization package in R built by combining data, aesthetics, and layers.

data + aesthetics + layers

Data = what you're plotting **aes()** = which variables control visual properties **geom_*()** = how the observations are drawn **scale_*()** = how values are displayed **facet_*()** = split into groups **labs()** = titles and labels **theme_*()** = appearance

2.7.1 Aesthetics

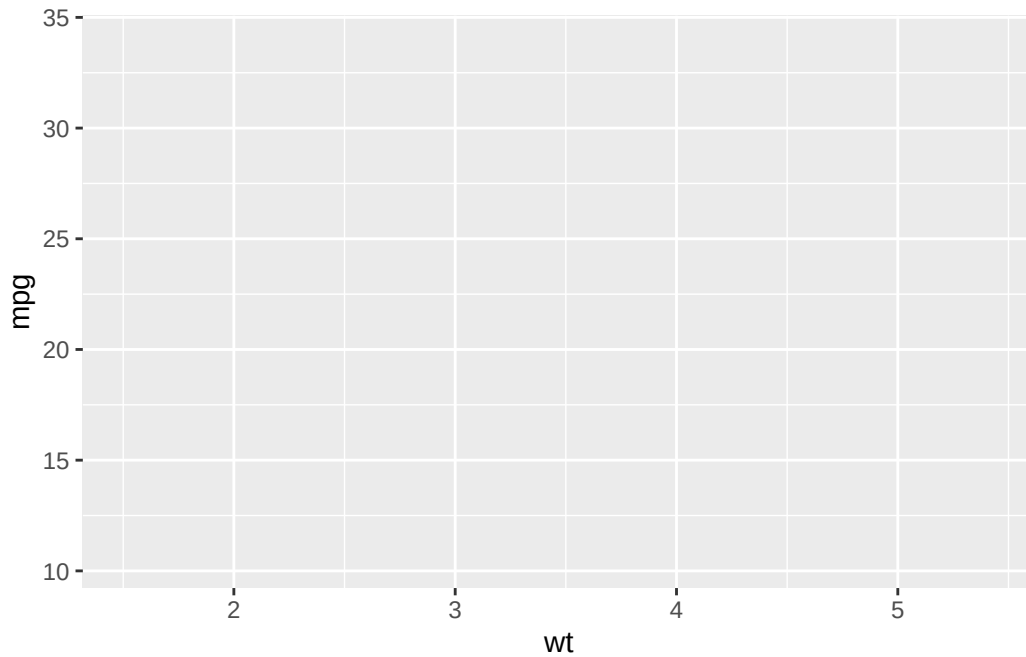
Aesthetics, specified with **aes()**, determine how variables in the data **are mapped to visual properties** of the graph.

Common aesthetics include:

- x — horizontal position
- y — vertical position
- color — color
- size — size

- shape — shape
- alpha — transparency

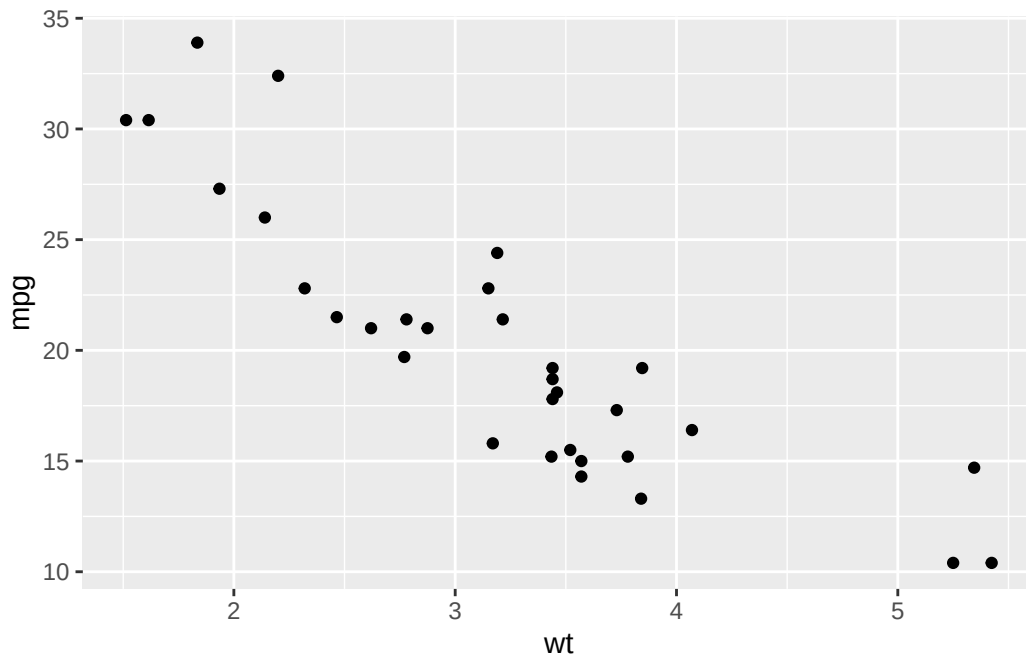
```
ggplot(mtcars, aes(  
  x = wt,  
  y = mpg,  
  color = factor(cyl)  
))
```



2.7.2 Layers

Layers determine how the data is actually displayed. They are usually added with `geom_*()` functions using `+`.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point()
```



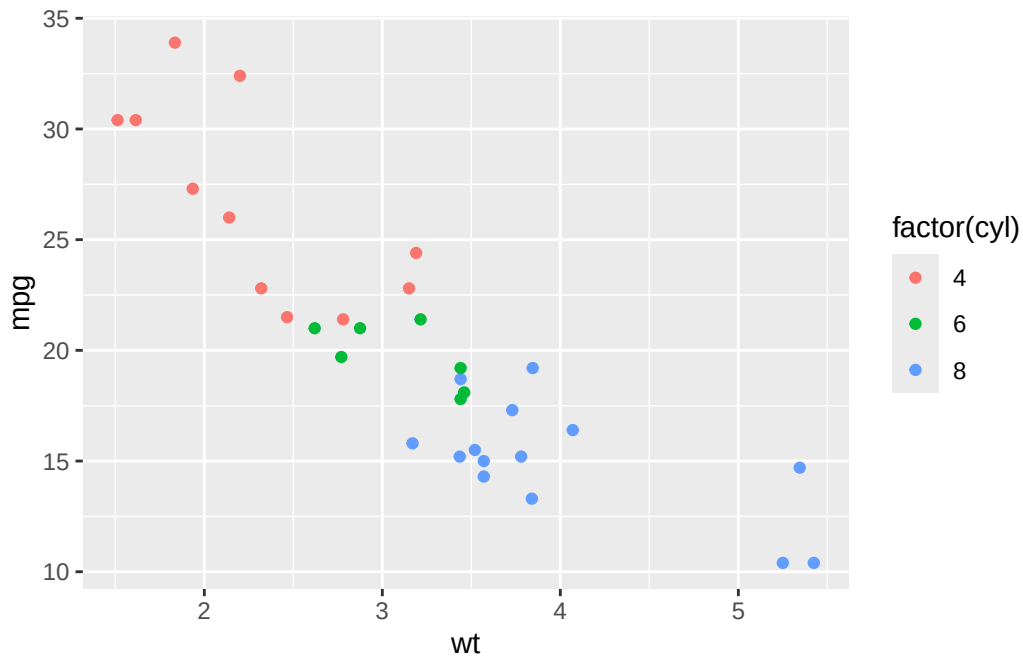
Examples include:

`geom_point()` `geom_histogram()` `geom_boxplot()` `geom_col()`

2.7.3 Mapping vs. Setting

Mapping means a visual property *varies* according to a variable in the data. Put it inside `aes()`

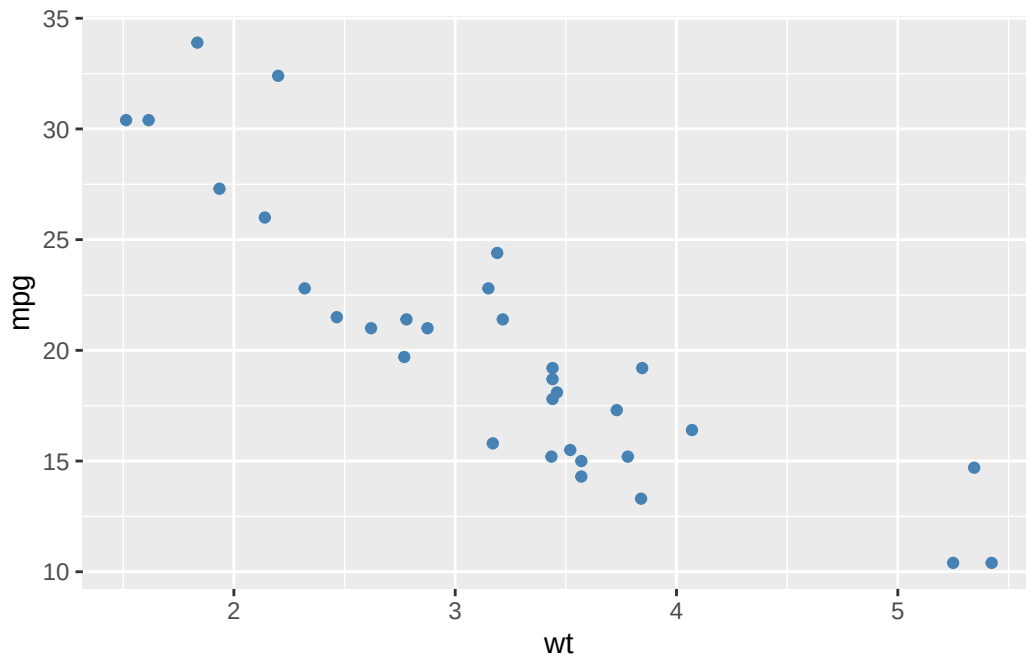
```
ggplot(mtcars, aes(  
  x = wt,  
  y = mpg,  
  color = factor(cyl)  
)) +  
  geom_point()
```



Different household-size categories receive different colors.

Setting means giving a visual property one fixed value. Put it outside `aes()`

```
ggplot(mtcars, aes(  
  x = wt,  
  y = mpg  
) +  
  geom_point(color = "steelblue")
```

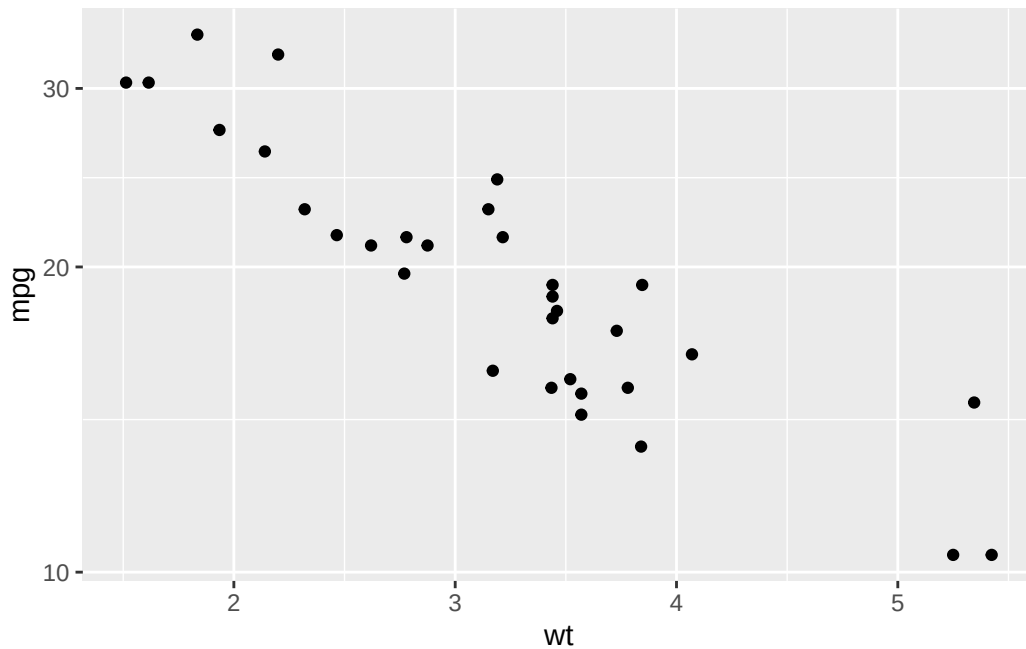


Now every point is steel blue.

2.7.4 Scales

Scales control how mapped data values are displayed, such as transforming or formatting an axis.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  scale_y_log10()
```

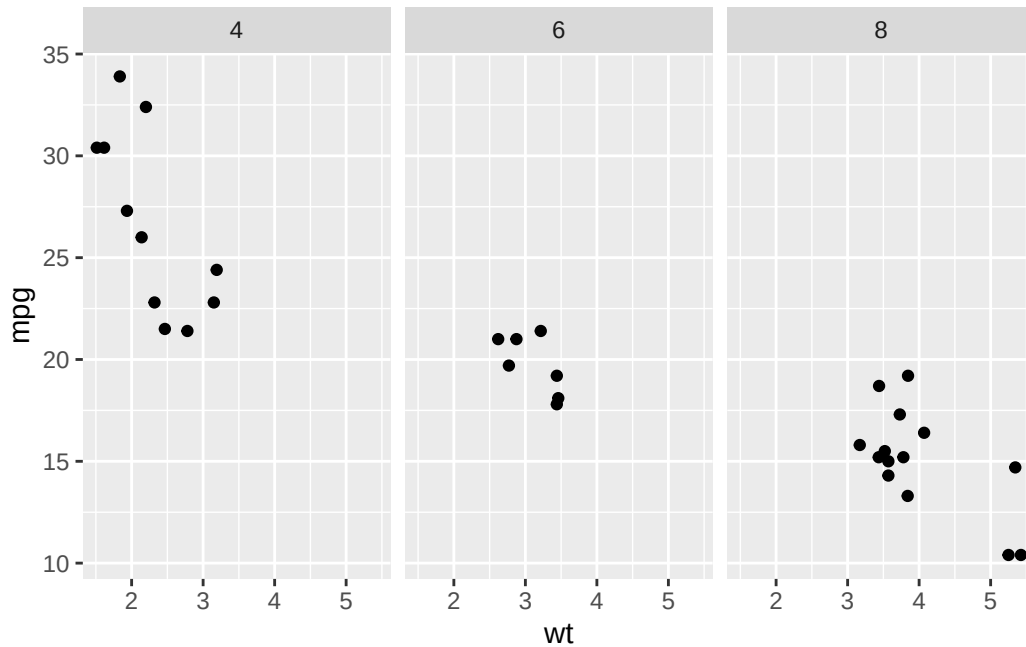


`scale_y_log10()` displays the y-axis on a logarithmic scale.

2.7.5 Facets

Facets split one visualization into separate plots for different groups.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  facet_wrap(~ cyl)
```

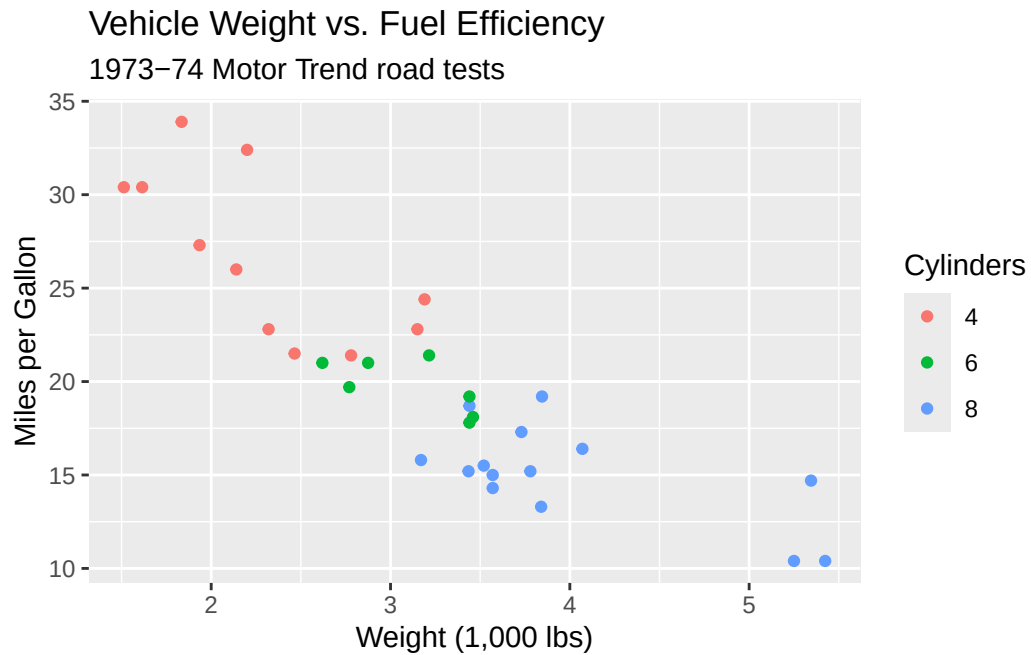


Instead of showing every household-size group on one graph, you get a separate panel for each household size.

2.7.6 Labels

`labs()` controls the descriptive text on the visualization:

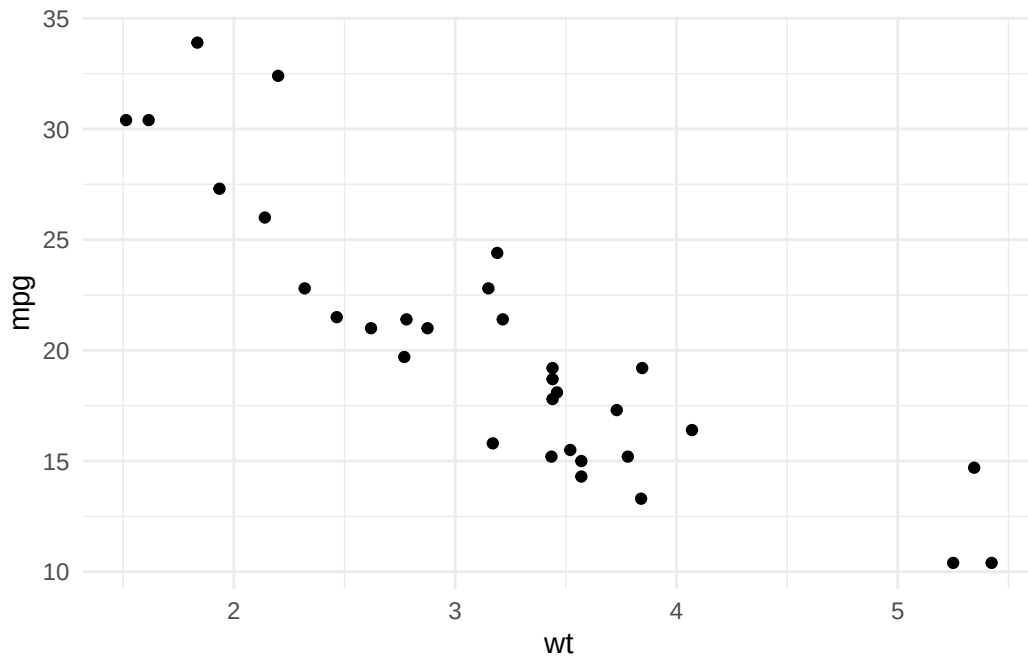
```
ggplot(mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point() +
  labs(
    title = "Vehicle Weight vs. Fuel Efficiency",
    subtitle = "1973-74 Motor Trend road tests",
    x = "Weight (1,000 lbs)",
    y = "Miles per Gallon",
    color = "Cylinders"
  )
```



2.7.7 Themes

Themes control the overall appearance of the plot rather than the underlying data.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  theme_minimal()
```



3 Descriptive Analytics for Time Series Data

3.1 Overview

For descriptive analysis, the focus was on questions such as: What is typical? How spread out is the data? What shape does the distribution have? With time series data, an additional question becomes important: When did it happen?

Time series data is a sequence of observations collected or recorded at successive points in time. Because observations are ordered chronologically, time series data provides a dynamic view of how a variable changes over time, rather than a static snapshot.

- Chronological order: Observations occur sequentially over time so ordering of the data matters.
- Time index: Each observation is associated with a specific point in time.
- Dependence over time: Earlier and later observations may be related, meaning observations may not be independent.
- Temporal patterns: The ordering of observations allows patterns like trend, seasonality, and short term variation to be identified.

Suppose a CFO says “Sales dropped 15% from Q3 to Q4.”

That number by itself doesn’t necessarily mean the business suddenly got worse. Maybe sales normally decline from Q3 to Q4 and this is just a regular reoccurring observation.

With time series data, there are three things to be distinguished:

- Trend → Is there a long-term upward/downward movement?
- Seasonality → Does a pattern repeat at regular intervals?
- Noise → Is a movement just short-term random variation?
- Level Change → Did the series actually shift to a different level?

3.2 Working with Lubridate

Why do dates matter in analytics? Dates can be used for several things, for example:

- Time aware filtering (e.g. time based filtering), time aware grouping (group a time period), and trend/seasonality detection.

- Aligning events (comparing before and afters).
- Preparing for forecasting.

Dates need to actually be dates! A date stored as text like:

```
'2025-08-21'
```

is much different from an R `Date` object actually representing August 21, 2025. The main package used when working with dates is `lubridate`.

3.2.1 Parsing Dates

Parsing dates is the process of converting raw string dates to an actual date/time object, which can then be converted into useful time components. Raw string dates may not always be written in a consistent way, therefore different functions are used based on what the text format is.

- `ymd()` when string = YEAR → MONTH → DAY
- `mdy()` when string = MONTH → DAY → YEAR
- `dmy()` when string = DAY → MONTH → YEAR
- `ymd_hms()` when string = YEAR → MONTH → DAY : HH:MM:SS

Example:

```
ymd("2025-08-21")
```

```
[1] "2025-08-21"
```

```
mdy("08/21/2025")
```

```
[1] "2025-08-21"
```

```
dmy("21-Aug-2025")
```

```
[1] "2025-08-21"
```

```
ymd_hms("2025-08-21 14:30:00")
```

```
[1] "2025-08-21 14:30:00 UTC"
```

Once the date is in the correct format, R can extract pieces:

- `year(x)` = extract year
- `month(x)` = extract month
- `quarter(x)` = extract quarter
- `wday(x)` = extract weekday
- `hour(x)` = extract hour
- `isoweek(x)` = extract week number of year

```
x <- ymd_hms("2025-08-21 14:30:00")  
x
```

```
[1] "2025-08-21 14:30:00 UTC"
```

```
year(x)
```

```
[1] 2025
```

```
month(x)
```

```
[1] 8
```

```
quarter(x)
```

```
[1] 3
```

```
wday(x)
```

```
[1] 5
```

```
hour(x)
```

```
[1] 14
```

```
isoweek(x)
```

```
[1] 34
```

Essentially the process is **raw date string** → **actual date/time object** → **useful time components**

Dates can also be created from combining variables that already exist. For example if you have `year = 2025` and `month = 8` you can construct an actual date.

`make_date()` use when year/month columns exist because it avoids rounding problems associated with fractional-year representations

```
# Before: year and month are separate variables
txhousing |>
  select(year, month) |>
  head(2)
```

```
# A tibble: 2 x 2
  year month
<int> <int>
1  2000     1
2  2000     2
```

```
# Create an actual date from year and month
TX <- txhousing |>
  mutate(
    date_actual = make_date(
      year = year,
      month = month,
      day = 1
    )
  )

# After: year and month have been combined into a date
TX |>
  select(year, month, date_actual) |>
  head(2)
```

```
# A tibble: 2 x 3
  year month date_actual
<int> <int> <date>
1  2000     1 2000-01-01
2  2000     2 2000-02-01
```

3.2.2 Calendar Arithmetic

Leap years can screw up calculation of dates. For example `ymd("2024-02-29")` is a leap year. If you subtract five years you'd get 2019-02-29 which doesn't exist.

```
d1 <- ymd("2024-02-29")
```

lubridate has calendar awareness operators:

```
d1 %m-% years(5)
```

```
[1] "2019-02-28"
```

```
ymd("2023-01-31") %m+% months(1)
```

```
[1] "2023-02-28"
```

3.2.3 Time Windows

Once dates are represented properly, normal comparisons work:

```
TX |>
  filter(
    date_actual >= as.Date("2005-01-01"),
    date_actual <= as.Date("2010-12-31")
  )
```

```
# A tibble: 3,312 x 10
```

	city	year	month	sales	volume	median	listings	inventory	date	date_actual
	<chr>	<int>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<date>
1	Abilene	2005	1	119	1.03e7	74200	570	3.7	2005	2005-01-01
2	Abilene	2005	2	130	1.18e7	74300	581	3.7	2005.	2005-02-01
3	Abilene	2005	3	170	1.45e7	71300	661	4.3	2005.	2005-03-01
4	Abilene	2005	4	171	1.64e7	80300	650	4.2	2005.	2005-04-01
5	Abilene	2005	5	176	1.81e7	87100	660	4.2	2005.	2005-05-01

```

 6 Abilene  2005      6   230  2.41e7  92500      664      4.1 2005. 2005-06-
01
 7 Abilene  2005      7   177  1.85e7  93500      657      4.1 2006. 2005-07-
01
 8 Abilene  2005      8   216  2.41e7  97500      638      3.9 2006. 2005-08-
01
 9 Abilene  2005      9   165  1.71e7  92800      644      3.9 2006. 2005-09-
01
10 Abilene  2005     10   161  1.64e7  92300      601      3.6 2006. 2005-10-
01
# i 3,302 more rows

```

You can also make relative windows.

For example, find the most recent date:

```

max_date <- max(TX$date_actual, na.rm = TRUE)
max_date

```

```
[1] "2015-07-01"
```

Then go five years backward:

```

start_5y <- max_date %m-% years(5)
start_5y

```

```
[1] "2010-07-01"
```

Then:

```

recent <- TX |>
  filter(date_actual > start_5y)

head(recent)

```

```

# A tibble: 6 x 10
  city      year month sales  volume median listings inventory  date date_actual
<chr>  <int> <int> <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl> <date>
1 Abilene  2010     8   144 17504580 122000     936     6.7 2011. 2010-08-
01

```

2	Abilene	2010	9	116	15475763	121300	899	6.5	2011.	2010-09-01
3	Abilene	2010	10	111	14570529	111900	863	6.4	2011.	2010-10-01
4	Abilene	2010	11	115	11422741	87900	851	6.4	2011.	2010-11-01
5	Abilene	2010	12	116	15289470	118300	830	6.3	2011.	2010-12-01
6	Abilene	2011	1	68	8834493	123300	809	6.1	2011	2011-01-01

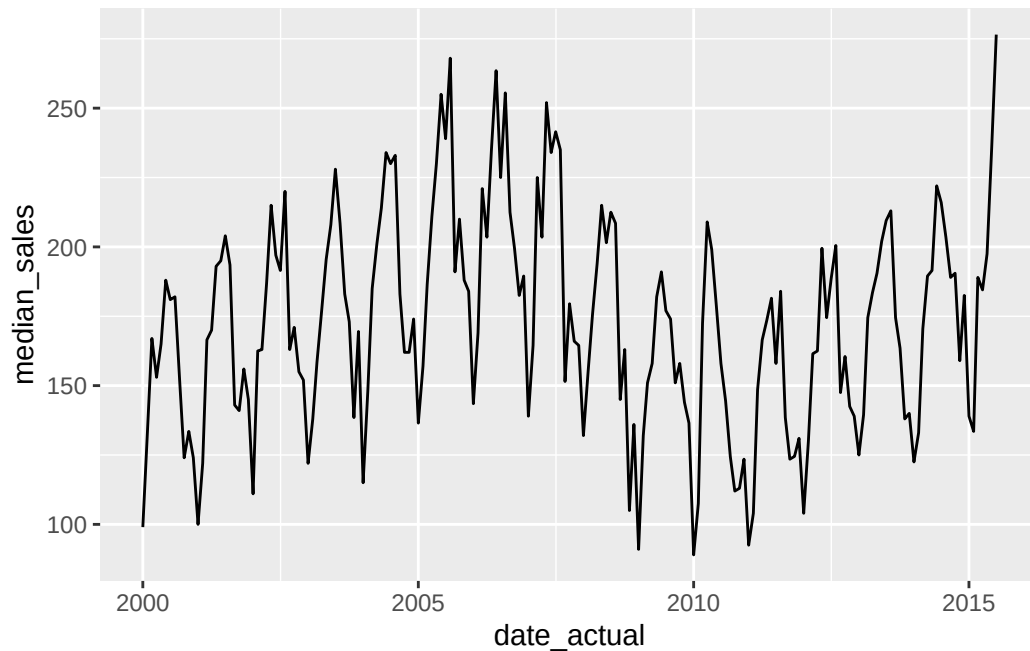
So instead of hard-coding a five-year range, you're saying:

Give me observations from the final five years of the dataset.

For time-series data, the standard visualization is a line chart.

The lecture calculates statewide median sales for each month:

```
TX |>
  group_by(date_actual) |>
  summarise(
    median_sales = median(sales, na.rm = TRUE)
  ) |>
  ggplot(aes(date_actual, median_sales)) +
  geom_line()
```



The conceptual structure is:

- Date → x-axis
- Value → y-axis

Connecting observations makes the evolution of the variable visible.

`scale_x_date()` then controls how frequently dates are displayed so that a long time horizon doesn't produce an unreadable x-axis.

3.2.4 Seasonality

Seasonality is one of the central concepts of time series data.

Seasonality is a pattern that systematically repeats according to the calendar.

It can be viewed by:

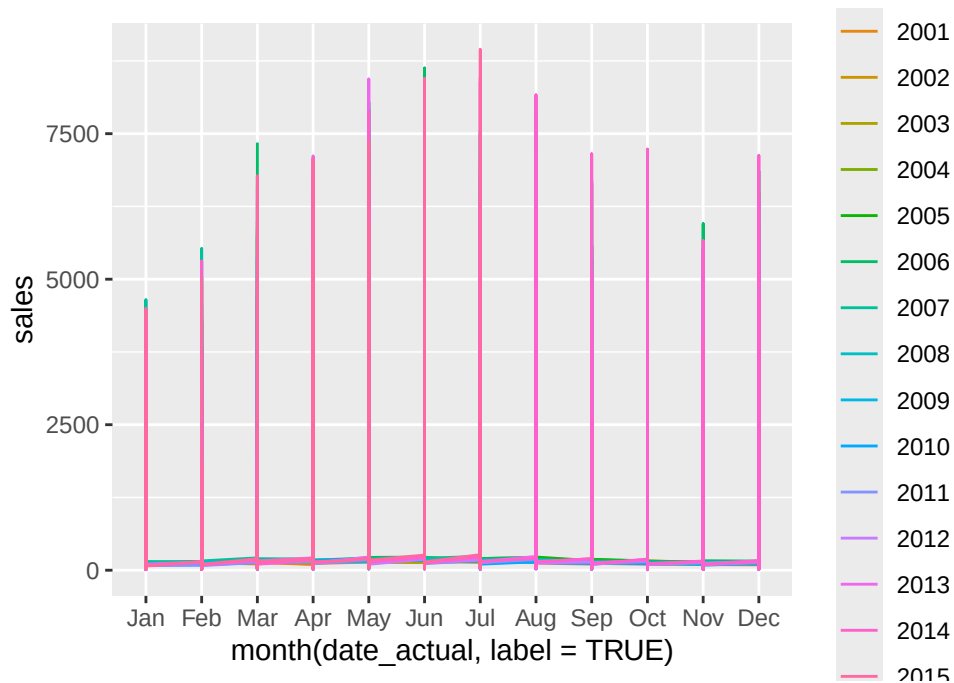
Lines by year:

```
TX |>
  ggplot(aes(
    x = month(date_actual, label = TRUE),
    y = sales,
    group = year,
```

```

    color = as.factor(year)
  )) +
  geom_line()

```



Boxplots by month

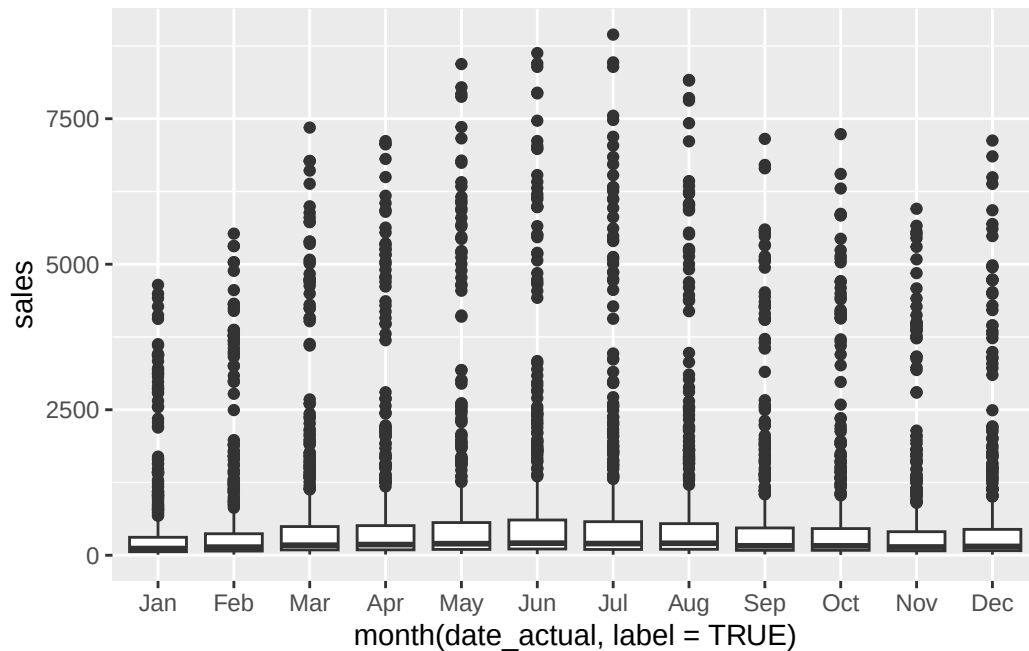
What does a typical January look like across years? What does a typical July look like?

```

TX |>
  ggplot(aes(
    x = month(date_actual, label = TRUE),
    y = sales
  )) +
  geom_boxplot()

```

Warning: Removed 568 rows containing non-finite outside the scale range (``stat_boxplot()``).



Heatmaps are another view of seasonality

2005 2006 2007 2008 ...

Jan Feb Mar ... Dec

where the fill intensity represents sales.

3.2.5 Moving Averages

Month-to-month fluctuations can make the underlying trend difficult to see. A rolling average smooths those fluctuations.

```
monthly_sales <- TX |>
  group_by(date_actual) |>
  summarise(
    median_sales = median(sales, na.rm = TRUE)
  ) |>
  arrange(date_actual) |>
  mutate(
    moving_average = rollmean(
      median_sales,
```

```

    k = 12,
    fill = NA,
    align = "right"
  )
)
monthly_sales

```

```

# A tibble: 187 x 3
  date_actual median_sales moving_average
  <date>         <dbl>         <dbl>
1 2000-01-01         99          NA
2 2000-02-01        134          NA
3 2000-03-01        167          NA
4 2000-04-01        153          NA
5 2000-05-01        165          NA
6 2000-06-01        188          NA
7 2000-07-01        181          NA
8 2000-08-01        182          NA
9 2000-09-01        152          NA
10 2000-10-01        124          NA
# i 177 more rows

```

$k = 12$ means a 12-observation window. Because the data are monthly, that's a 12-month moving average.

4 Probability Rules and Distributions

So far, we've covered descriptive analytics, where tools and methods are used to describe

5 Lecture 5

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

6 Lecture 6

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

7 Lecture 7

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

8 Lecture 8

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

9